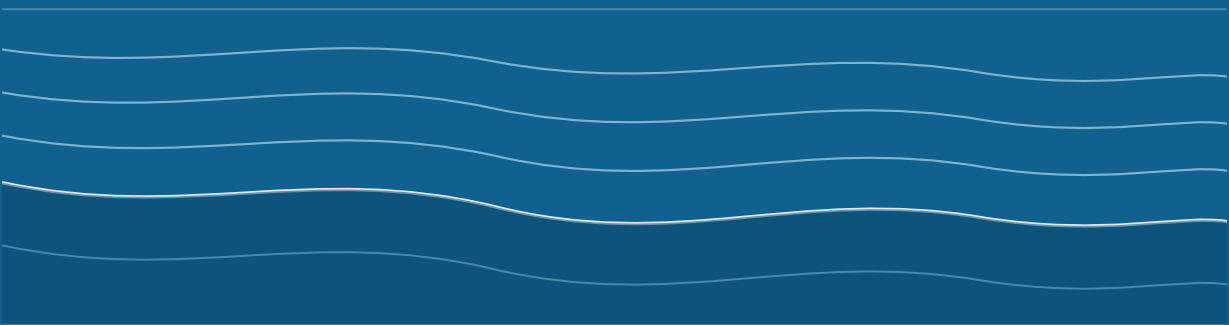


Jaipur Flood Vulnerability Index

How the model works, why every choice was made, and how to defend all of it in a conversation.



What is in here

Read it start to finish once. After that, Part 10 alone is enough to revise from.

01 The one idea everything rests on

Water is stupid. It only knows one rule.

02 Where the data comes from

Height hidden inside colours, a map made by volunteers, ten years of rain.

03 The hydrology — the actual science

Fill the holes, route the flow, count the water, measure the wetness.

04 Turning maps into a table

A chessboard over Jaipur, and the mistake that stretches your city sideways.

05 Scoring

Why we rank instead of scale, and what that costs.

06 Testing your own assumptions

The part almost nobody does. Two hundred runs with the weights shuffled.

07 How you would know if it were wrong

Ground truth, why you need the dry places, and what AUC really means.

08 The bug worth talking about

A correct answer that was completely useless.

09 What it cannot do

Five limits, including the one that is still unsolved.

10 The questions you will be asked

Five likely questions, with answers, and the sixty-second summary.

The one idea everything rests on

Water is stupid. It only knows one rule: go downhill.

That is the whole insight. Everything else in this project is detail hung off it.

Water does not know which colony is wealthy. It does not know where the drain *should* have been built. It rolls downhill, and when it reaches somewhere it cannot roll out of, it stops there and waits.

So if you know the **shape of the ground**, you can work out where water will end up — before a single drop has fallen.

Why this matters in Jaipur specifically

Every monsoon, certain roads and colonies go under water. Mostly the same ones, year after year. The Municipal Corporation has a limited drainage budget, so it cannot fix everywhere, and it has to choose.

That choice is usually made from complaint volume and institutional memory. Which has an obvious flaw: **it favours the neighbourhoods that complain loudest, not the ones worst affected**. An area where people have the time, the phone numbers and the confidence to call the office gets attention. An area where they do not, does not.

THE POINT OF THE PROJECT

Water does not care who complains. Asking the landscape where the water goes gives you a way of deciding where the drainage money goes that is not based on who shouts.

Where the data comes from

Three sources. All free, none needs an account. That is deliberate — a model nobody else can run is not much of a model.

The height map, and a genuinely clever trick

You need to know how high every point in Jaipur is. In the year 2000, NASA flew the Space Shuttle around the Earth with radar pointing downwards and measured the height of nearly all the land on the planet. That data is public.

But the interesting part is *how it is stored*.

A photo is made of pixels, and each pixel holds three numbers: how much red, how much green, how much blue. Each runs from 0 to 255.

Someone realised that $256 \times 256 \times 256$ is about **16.7 million** combinations — far more than a picture needs. So why not use those three numbers to store a *height* instead of a colour?

THE DECODING FORMULA

$$\text{height in metres} = (\text{red} \times 256) + \text{green} + (\text{blue} \div 256) - 32768$$

Open one of these files and it looks like meaningless green sludge. Run that formula over every pixel and you have the height of the ground, in metres, across all of Jaipur — roughly one number every 30 metres.

WHY SUBTRACT 32768?

So the format can store places below sea level, like the Dead Sea. Without that offset you could never represent a negative height.

The city itself

Height tells you about the land. You also need the city, and that comes from **OpenStreetMap** — a map of the world built by volunteers, like Wikipedia but for maps. Free to download.

LAYER	WHAT IT TELLS YOU
Drains, nalas, rivers	Where water can escape
Roads	How built-up an area is — and whether they are paved or dirt
Buildings	Where people actually are
Schools, clinics, markets	Places that hurt more when they flood

The rain

Ten years of real measured daily rainfall for Jaipur, free, from a weather archive. This gives you one number that matters enormously:

THE NUMBER TO REMEMBER

About 85% of Jaipur's rain falls between June and September.

Think about what that means. If you size drainage for the *average* rainfall, you have built drainage that fails every single year — because rain does not arrive as an average. It arrives all at once.

The hydrology — the actual science

Four steps. The order is not optional — each one needs the one before it. Run them out of order and you produce a map that looks beautiful and means nothing at all.

Step 1 — Fill the holes

The problem. Satellite data contains mistakes. Sometimes one pixel comes out two metres lower than it really is. Just an error.

That fake pit is a serious problem. Water flowing into it has nowhere to go, because every neighbour is uphill. In your calculation the water reaches that pixel and *vanishes* — and everything downstream is now wrong, because water that should have arrived never did.

PICTURE IT

Imagine Jaipur is a sandpit with lots of small holes in it. Start pouring water in from the edges, slowly.

Water fills the lowest hole first. It rises. When it reaches the rim it spills over into the next area. Keep going.

At the end every hole is filled to exactly the level at which water could escape. The hills are untouched, because the water never got that high.

That is the algorithm, and it is called **Priority-Flood**. "Priority" because at every moment you always deal with the *lowest* part of the water's edge first.

THE FREE BONUS

Subtract the original height from the filled height and you get something genuinely useful: **how deep a hole each place sits in**. A spot in the middle of a bowl comes out at three metres. A spot on a ridge comes out at zero.

Those three-metre spots are places water physically cannot escape from. That is a flood map on its own.

Step 2 — Which way does the water go?

For every square on the map, ask: **if I put a drop of water here, which way does it roll?**

A square has eight neighbours around it, like a king on a chessboard. The water goes to whichever one is steepest downhill. This is called **D8** — D for direction, 8 for the eight choices.

THE TRAP THAT CATCHES EVERYONE

Diagonal neighbours are further away. If a square is 30 m across, the neighbour directly north is 30 m away — but the neighbour to the north-east is 42 m away ($30 \times \sqrt{2}$).

So you must compare **steepness**, not **drop**. A 3 m drop over 30 m is steeper than a 3 m drop over 42 m.

Forget this and every flow path in your entire map leans diagonally. And here is the evil part: **it does not crash**. You get a beautiful map that is wrong. Worth mentioning if someone asks what was tricky.

Step 3 — Count the water

The question: how much land drains through each square?

THINK OF A SCHOOL CORRIDOR

Standing in a corner beside one classroom, maybe five people walk past you. Standing at the main staircase, five hundred do. Same corridor, completely different situation.

Water is the same. A square on a hill catches only its own rain. A square at the bottom of a valley catches rain from *everything above it*.

The counting trick. Sort every square from highest to lowest, then work through them in that order. By the time you reach any square, every square that drains into it has already been handled — so you simply take your running total and pass it downstream.

One pass. No going back, no getting stuck in loops. Every square starts at 1 (its own rain). By the end a square might hold 50 — or 5,000. That is the difference between "catches its own rain" and "is effectively a river".

THE MOMENT THAT SHOULD GIVE YOU A SMALL THRILL

You never told the computer where Jaipur's rivers and nalas are. But draw this map and a branching network appears that looks exactly like a river system. The computer worked out the city's drainage purely from the shape of the ground.

Compare it against a satellite photo. It matches. That is your proof the maths is working.

Step 4 — The wetness index

The single most useful number in the whole project. It looks frightening. It is not.

$$TWI = \ln \frac{a}{\tan \beta}$$

Beven & Kirkby, 1979 — still the standard after forty-five years

THINK OF A BATHTUB

How much water sits in a bath depends on exactly two things:

- how fast water pours **in** — the tap
- how fast it drains **out** — the plughole

Now map that onto the formula:

- ***a* on top** is how much water arrives (the tap) — the flow accumulation from Step 3
- ***tan β* on the bottom** is how steep the ground is (the plughole). Steep ground drains fast; flat ground drains slowly.

So the fraction gets large when two things are true at once:

- lots of water arrives — **big top**
- the ground is flat so it cannot leave — **small bottom**

Somewhere that is both low and flat scores enormously. And that describes almost every place in any city that floods every single year. The `ln` at the front simply squashes a very wide range of numbers into something you can put on a colour scale. Nothing clever about it.

TWO BUGS HIDING IN ONE SMALL FORMULA

One. $\tan(0) = 0$, and dividing by zero gives infinity. Perfectly flat ground would fill your whole map with infinities. The fix is to never let the slope be exactly zero — force a tiny minimum.

Two. $\tan()$ needs **radians**, not degrees. Pass degrees and the code runs perfectly while every single answer is wrong. Silent. No error.

There is a test in the project written specifically for each of these.

Turning maps into a table

You now have four things that do not fit together at all, and statistics needs a spreadsheet.

What you are holding after Part Three:

- a grid of heights — millions of tiny pixels
- a list of squiggly lines — drains
- another list of squiggly lines — roads
- a scatter of dots — buildings

You cannot do maths on that. You need rows and columns.

The solution: draw a chessboard over Jaipur

Each square is 300 metres across. Then for every square, ask the same set of questions.

HOW HIGH	HOW FLAT	HOW WET	BUILDINGS	ROAD (M)	TO DRAIN (M)
412	0.8°	12.4	87	1,240	640

One row per square. That is your table, and everything after this point is ordinary data science of the kind you would do on any dataset. The step is called **rasterisation**, and honestly it is most of what real GIS work consists of.

WHY 300 METRES?

A trade-off. Smaller squares give more detail but are noisier, and your 30 m height data cannot really support them. Bigger squares lose the difference between one colony and the next. 300 m is the sweet spot.

The gotcha nobody warns you about

Lines of longitude get closer together as you move towards the poles. They all meet at the top.

So at the equator one degree of longitude is about 111 km. **In Jaipur it is only about 99 km.**

THE CLASSIC BEGINNER GIS MISTAKE

Ignore this and your entire map is **stretched sideways by 11%**. Every distance is wrong. Every density is wrong. And once again — nothing crashes.

The code handles it. Knowing that it needs handling is a good sign to anyone who knows the field.

Scoring

Where the honest thinking happens.

The problem

You cannot add a slope in degrees to a count of buildings. It is like adding 5 kilograms to 3 hours. Meaningless.

And if you added them anyway, whichever has bigger numbers would take over. Building counts run to 200; slopes run to 15. Buildings would drown out slope completely, for no reason except the units they happen to use.

So everything has to go onto the same 0-to-1 scale first.

The obvious way, and why it fails

The obvious approach is $(\text{value} - \text{smallest}) \div (\text{largest} - \text{smallest})$. Squash everything between 0 and 1. Here is why it breaks.

Flow accumulation is wildly lopsided. Most squares sit around 50. But a square on the Dravyavati channel might be 400,000.

Squash that. The 400,000 becomes 1.0 — and everything else in the entire city (50, 200, 900, 3,000) gets crushed into the bottom 1% of the scale. They all become effectively zero.

Your map would show **one bright line and a completely flat grey city**. Technically correct. Completely useless.

What we do instead: ranking

THINK OF EXAM MARKS

Someone gets 98%, someone gets 42%, someone gets 41%. Instead of using the marks, use the **class rank**: 1st, 2nd, 3rd.

That is what we do. Not "what is this square's flow accumulation?" but **"what fraction of Jaipur's squares are lower than this one?"**

A square in the worst 5% scores 0.95 — whether the very worst square is ten times the average or ten thousand times. Now the map actually shows differences everywhere.

THE COST — SAY THIS OUT LOUD

Ranking throws away *size*. The index can say "this is worse than 90% of Jaipur." It can **never** say "this place is genuinely in danger."

Everything is relative to Jaipur itself. Run this on a city that never floods and you would still get a top-twenty list — it would just be the least-dry parts of a dry city.

This makes it a tool for deciding what to look at first. Not a warning system. Use that sentence exactly when someone asks.

Then blend the three scores

$$\text{Risk} = (0.50 \times \text{Hazard}) + (0.25 \times \text{Exposure}) + (0.25 \times \text{Sensitivity})$$

COMPONENT	THE QUESTION IT ANSWERS	BUILT FROM
Hazard	Will water collect here?	Wetness index, depression depth, flow accumulation, distance to a drain, slope
Exposure	Is anything here to damage?	Buildings, roads, schools and clinics
Sensitivity	Can people here cope?	Unpaved road share, packed housing, no drains mapped nearby

WHY ADD, WHEN TEXTBOOKS SAY MULTIPLY?

The textbook formula is Hazard × Exposure × Vulnerability, and there is a good argument for it: an empty desert has zero exposure, so it should carry zero risk.

But multiplying is brittle. If *any* single layer is missing or zero, the whole score collapses to zero. Our data has gaps. Adding degrades gently — a gap weakens the score instead of destroying it.

We chose adding. It is a real trade-off, it is not hidden, and the reasoning is written into the code.

BE CAREFUL WITH THE SENSITIVITY LAYER

Ward-level income data for Jaipur is not public at this resolution, so this component uses infrastructure *proxies*. Proxies can be wrong about real neighbourhoods: a new development has unpaved roads without deprivation; the walled city is dense without being poor.

Describe these outputs as "*areas with infrastructure characteristics associated with disadvantage*", never as "poor areas". The distinction matters, because getting it wrong misdirects resources away from people who need them.

Testing your own assumptions

This is the part that will impress people most, and almost nobody does it.

Look at those weights again: 0.50 , 0.25 , 0.25 . **Where did they come from? You made them up.** Reasonable judgement — but a judgement.

So a sharp person will say:

THE CHALLENGE

"If you had picked 0.30 instead of 0.50 you would get a different map. So why should I believe this one?"

That is completely fair. And here is the thing — **most students argue back.** That is the wrong move.

What we do instead: we test it

Run the whole thing **200 times**. Each run, randomly jiggle every weight by up to $\pm 50\%$. Then for each square ask: **how often did it land in the worst 10%?**

RESULT

WHAT IT MEANS

198 / 200

That is real. It does not matter what weights you pick. You can defend it in any argument.

jumps around **That is your assumptions talking.** Reporting it as a finding would be overclaiming.

Only the stable squares go onto the priority list.

THE MOVE THAT SEPARATES THIS FROM A SCHOOL PROJECT

When someone challenges your weights, you do not argue. You say: *"I tested that. Here is what survives and here is what does not."*

How you would know if it were wrong

A colourful map looks convincing whether or not it is true. This is the most important thing to understand about the entire project.

You cannot skip this. And it is the one part nobody can do for you.

What you need

A list of real places in Jaipur, each with a source:

- places that **really flooded**
- places that **really stayed dry**

A source means a news article, a photo you took, a date, or a shopkeeper who told you. An observation without a source is an assertion, not evidence.

THE TRAP — YOU NEED THE DRY ONES

If your list contains only flooded places, then a model that declares *"the entire city is high risk"* scores **100%**. A useless model that tested perfectly.

The dry places are what make the test mean anything. They are harder to find, because nobody photographs a road that is fine.

Three ways to find dry places

- 1 **Look in the background of flood photos.** There is often a perfectly dry road behind the flooded one. That is two data points, not one.
- 2 **Streets nobody complained about** during a storm that generated plenty of complaints elsewhere.
- 3 **Walk a route during heavy rain and write down every junction** — wet and dry. This is the best data you will ever get, because you controlled how it was collected.

Aim for **twenty of each**, minimum.

How to measure it: AUC

THE DEFINITION, IN ONE SENTENCE

Pick one flooded place and one dry place at random. **Did your model give the flooded one a higher score?** Do that for every possible pair. The fraction you got right is your AUC.

AUC	WHAT IT MEANS
0.50	A coin flip. No signal at all.
0.65	Weak, but real.
0.75	Genuinely good for a tool built from free open data.
0.85	Strong.
1.00	Something is wrong. Go and look for a bug.

WHY NOT "ACCURACY"?

Because accuracy can be faked. If 90% of your list flooded, then saying "everything floods" scores 90%. AUC cannot be gamed that way, and it does not depend on where you set a cut-off.

Two more honesty checks the code performs

Error bars

With only forty places your AUC is not exact. Saying "AUC 0.78" pretends to a precision you do not have. Saying "**0.78, and it could plausibly be anywhere from 0.61 to 0.91**" is honest — and it instantly tells a reader your sample is small.

The baseline test — this one takes nerve

The code checks whether the full composite index actually beats a single simple number, such as *how low is this spot compared to its neighbours*.

If the simple number does just as well, **all the extra machinery is decoration** — and we report that rather than burying it.

REMEMBER THIS ONE LINE

A validated 0.71 beats an unvalidated 0.95 every single time. Anyone who knows what they are doing will trust you more for the honest smaller number.

The bug worth talking about

A correct answer that was completely useless.

The first time the priority list ran, the top ten places all contained **two or three buildings**.

The maths was entirely correct. They were genuinely low, flat, water-collecting spots.

But they were **empty fields**.

Nobody needs an empty field drained. The list was hydrologically right and operationally worthless.

Why it happened

Because the three scores are *added* rather than multiplied, a square can score very high on hazard alone while having almost no exposure. That is the trade-off from Part Five showing its teeth.

The fix

The priority list now requires a real number of buildings. The map still shows every square — you never hide data — but the *action list* only includes places where people actually are.

TELL PEOPLE THIS STORY

It shows you looked at your own output, did not like it, understood why, and fixed it with the reasoning written down. That is worth far more in a conversation than a clean result with no story behind it.

What it cannot do

Say these before anyone asks. Volunteering your own weaknesses is the strongest thing you can do in a technical conversation.

- 1 **It is relative, not absolute.** A score of 95 means "worse than 95% of Jaipur", not "the water will be knee-deep here".
- 2 **300 metres cannot see a blocked drain.** A great deal of real flooding is one clogged inlet on an otherwise fine street. This model finds where the *land* is against you; it will miss the drain entirely.
- 3 **The height data is from the year 2000.** Jaipur has been dug up, built on and re-levelled for twenty-five years since. All of that is invisible to the model.
- 4 **No pipes.** We know a drain exists. We do not know how wide it is, what condition it is in, or whether it is full of plastic. That matters enormously.
- 5 **The one that genuinely bothers me.** See below — it deserves its own space.

THE UNSOLVED PROBLEM

OpenStreetMap is made by volunteers. Volunteers map the areas they live in and care about — which tends to mean better-off areas.

So a poor, badly-served area might show up as having *no drains mapped*, which the model reads as **risky**. But it may *also* show fewer buildings and fewer roads, which the model reads as **less exposed**.

Those pull in opposite directions, and I do not know which one wins. The model may be quietly worse at spotting risk in exactly the neighbourhoods it exists to help.

This is not solved. Saying so plainly is the honest position, and it is a better answer than pretending otherwise.

The questions you will be asked

Five likely questions, with answers you can actually stand behind.

"Is this machine learning?"

No, and I am careful about that. There is no labelled flood data for Jaipur, so there is nothing to train on. It is a weighted index built from known hydrology, which is what flood researchers actually use when labels are scarce. There is an optional machine learning layer once I have enough real observations — but the honest test is whether it beats the physics-based version, and often it will not.

"Where did your weights come from?"

I chose them. So I tested them — two hundred runs with randomly shuffled weights. I only report the places that stay high risk regardless of the weighting.

"How accurate is it?"

Right now It is not validated yet, so I am not quoting a number. Here is how I am collecting ground truth.

Later AUC of X, with a confidence interval of Y to Z, on N observations I collected myself.

"What was the hardest part?"

Realising that a correct map can still be useless. My first priority list was all empty fields.

"What would you do next?"

Radar satellite imagery from just after a big storm. Radar sees through cloud, and standing water shows up dark. That would give me real observed flooding across the whole city — and turn this into genuine machine learning with real labels.

THE SIXTY-SECOND VERSION

Water goes downhill and pools where the ground is low and flat. I used free NASA height data to work out where that happens across Jaipur, layered on OpenStreetMap data for what is built there and how well it is served, and scored every 300-metre square. Then I tested whether the answer survives changing my own assumptions, and I am validating it against places that actually flooded. It is a screening tool — it tells you which twenty streets to walk first, not how deep the water will be.

The numbers worth memorising

Grid resolution	300 m squares
Height data	NASA SRTM, ~30 m per pixel, surveyed in 2000
Rainfall record	Ten years, ~85% falling June to September
Index weights	Hazard 0.50, Exposure 0.25, Sensitivity 0.25
Robustness test	200 runs, weights shuffled by up to $\pm 50\%$
Validation target	20 flooded and 20 dry observations, minimum
Metric	AUC, with a bootstrap confidence interval
Cost to run	Nothing. No API keys, no accounts.